

POST-DEPLOYMENT APPLICATION ASSESSMENT

Project: Task Manager Application (v2.0)

Date: 2026-07-13

Author: Do Ngoc Tung

Language: English

1 Executive Summary

1.1 Purpose of Assessment

This document provides a comprehensive post-deployment assessment of the Task Manager Application. The system is architected as a static Single Page Application (SPA) built with Next.js 16.2.9, deployed via static export, and completely decoupled from the AWS serverless backend. The frontend communicates with AWS services through authenticated API calls to AWS API Gateway and leverages AWS Amplify v6 for Cognito-based authentication.

The primary objective of this assessment is to evaluate the system's architectural integrity, validate performance metrics, and verify post-deployment security controls—specifically API throttling and cost-optimization mechanisms—ensuring the application meets production-grade reliability standards.

1.1.1 Verified Technologies

Category	Technology / Service	File / Verifiable Evidence
Framework	Next.js 16.2.9	package.json
Framework	React 19.2.4	package.json
Language	TypeScript ^5	package.json
Auth / Cloud	AWS Amplify ^6.18.0	package.json, lib/aws-config.ts
Auth / Cloud	Amazon Cognito User Pool	lib/aws-config.ts (userPoolId, userPoolClientId)
Auth / Cloud	AWS API Gateway	services/taskService.ts (NEXT_PUBLIC_API_URL)
UI / Styles	Tailwind CSS ^4	package.json, components.json
UI / Styles	@tailwindcss/postcss ^4	package.json
UI / Components	shadcn/ui ^4.13.0	package.json, components.json
UI / Components	@base-ui/react ^1.6.0	package.json

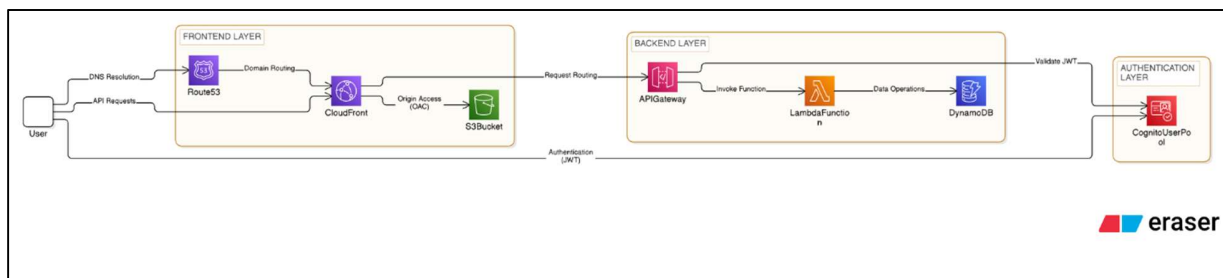
UI / Icons	lucide-react ^1.23.0	package.json, components.json
UI / Utilities	clsx ^2.1.1	package.json
UI / Utilities	class-variance-authority ^0.7.1	package.json
UI / Utilities	tailwind-merge ^3.6.0	package.json
Notifications	sonner ^2.0.7	package.json
Animations	tw-animate-css ^1.4.0	package.json

2 Architecture & Data Flow

2.1 System Architecture Diagram

Architecture Overview:

- Static Content Delivery: User requests are resolved via Route 53 and routed to Amazon CloudFront. CloudFront serves static assets (Next.js export) securely from a private S3 Bucket using Origin Access Control (OAC), ensuring the bucket is inaccessible from the public internet.
- Dynamic API Routing: Frontend API calls are securely routed through Amazon API Gateway.
- Authentication & Compute: API Gateway relies on Amazon Cognito for JWT validation. Authenticated requests are forwarded to AWS Lambda (business logic layer), which performs standard CRUD operations on Amazon DynamoDB (data persistence layer).

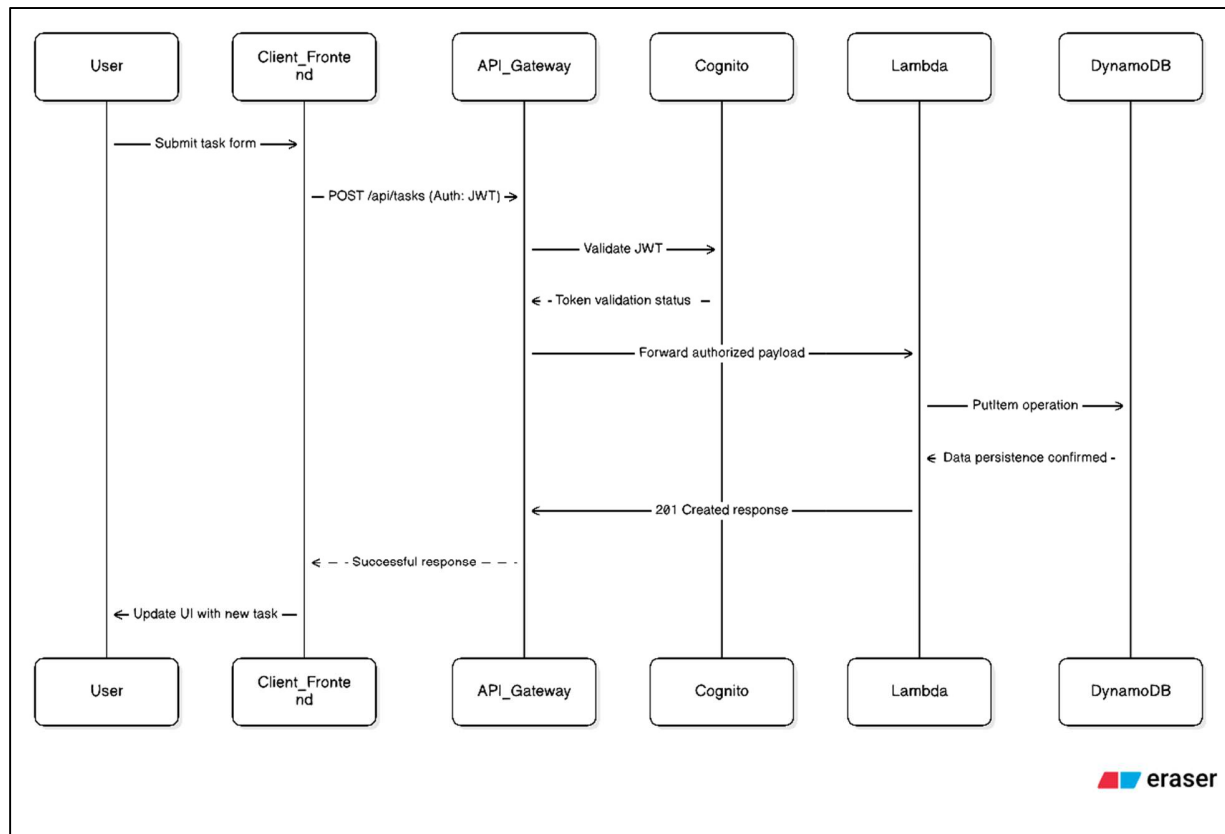


2.2 Data Flow Diagram

Step-by-step Sequence:

- Submit Request: The client frontend compiles task data and triggers a POST /api/tasks request, attaching the Cognito-issued JWT in the Authorization header.
- Authentication: API Gateway intercepts the request and validates the JWT signature and expiration against the Cognito User Pool.
- Forward Payload: Upon successful validation, API Gateway forwards the authorized payload (including the extracted user_id from the token claims) to the Lambda function.

- Data Persistence: The Lambda function processes the business logic and executes a PutItem operation to Amazon DynamoDB.
- Response Handling: DynamoDB confirms the insertion. Lambda returns an HTTP 201 (Created) status to API Gateway, which proxies the successful response back to the client to update the UI.



2.3 Configuration & Environment Variables

2.3.1 Core AWS Service Configurations

AWS Service	Configuration Details
AWS Lambda	Runtime: <code>nodejs22.x</code> , Memory: 128 MB, Timeout: 3s
Amazon DynamoDB	NoSQL, Billing: On-Demand
Amazon API Gateway	REST API, Edge-optimized
Amazon Cognito	User Authentication
CloudFront	Distribution: <code>d2qbjxgo2qsmoa.cloudfront.net</code>

Amazon S3	Static Web Hosting Origin
Route 53	Custom Domain Management
AWS IAM	Lambda Execution Role
AWS CloudWatch	Log Group

2.3.2 Application Environment Variables (Frontend)

Environment Variable Name	Target Component	Purpose / Description
NEXT_PUBLIC_AWS_REGION	AWS Amplify / SDK	Specifies the AWS Region hosting the Cognito and API Gateway resources.
NEXT_PUBLIC_AWS_COGNITO_USER_POOL_ID	Amazon Cognito	The unique ID of the Cognito User Pool used by AWS Amplify to manage user authentication flows.
NEXT_PUBLIC_AWS_COGNITO_APP_CLIENT_ID	Amazon Cognito	The App Client ID required to authenticate application requests without exposing a client secret.
NEXT_PUBLIC_API_URL	API Gateway Client	The base invoke URL of the Amazon API Gateway REST API endpoint.

2.3.3 Application Environment Variables (Backend)

Environment Variable Name	Target Component	Type / Availability	Purpose / Description
DYNAMODB_TABLE_NAME	AWS Lambda Configuration	Manually Configured	Specifies the target DynamoDB table name so the function knows where to execute CRUD operations.

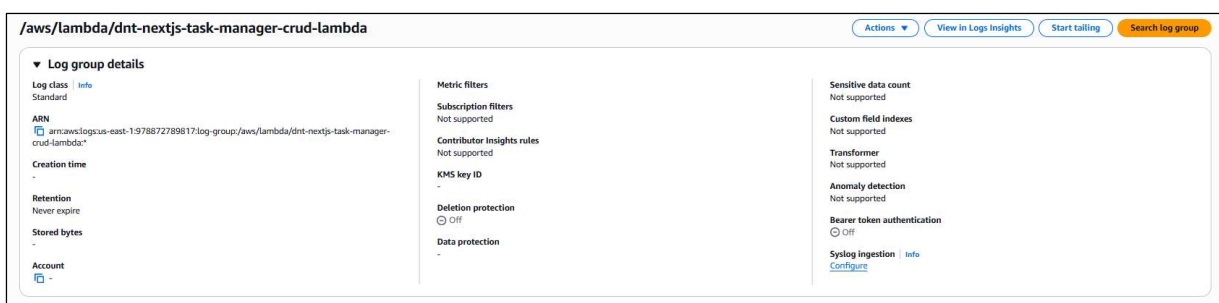
AWS_REGION	AWS Lambda Runtime	System Injected (Implicit)	A reserved runtime variable automatically provided by AWS. The AWS SDK implicitly reads this value to locate and authenticate requests to the DynamoDB endpoint without manual configuration.
------------	--------------------	----------------------------	---

3 Observability & Operations

3.1 Logging & Tracing Strategy

3.1.1 AWS Lambda Execution Logs

- Mechanism: All runtime outputs, including application errors, system traces, and standard execution logs generated by the `dnt-nextjs-task-manager-crud-lambda` function, are automatically captured and routed to Amazon CloudWatch Logs.
- Configuration Details: The dedicated log group is explicitly designated as `"/aws/lambda/dnt-nextjs-task-manager-crud-lambda"` within the `"us-east-1"` region.
- Retention Policy: The log retention period is currently configured to Never expire, ensuring full historical traceability of serverless execution states and backend API interactions for long-term audit purposes.



3.1.2 Amazon API Gateway Logs

- Status: Both Execution Logging (CloudWatch Logs) and Access Logging are currently Inactive (Disabled) for the dev stage.

- Operational Impact: The API Gateway layer does not generate independent transit logs or client access metrics in CloudWatch. Monitoring and troubleshooting of incoming API mutations rely entirely on the backend Lambda execution traces and application-level logging.

3.1.3 Distributed Tracing (AWS X-Ray)

- Status: X-Ray Tracing is currently Inactive (Disabled) on the API Gateway configuration.
- Operational Impact: End-to-end distributed tracing across the serverless execution path (Client -> API Gateway -> Lambda -> DynamoDB) is not activated. Request lifecycles are analyzed linearly using time-stamped correlation IDs within CloudWatch.

3.2 Backup & Disaster Recovery

Amazon DynamoDB Data Protection

- Table Name: dnt-nextjs-task-manager-tasks-table
- Point-in-Time Recovery (PITR): Currently Disabled. The system does not maintain continuous, incremental backups for granular per-second restoration.
- Deletion Protection: Currently configured to Off. The table can be deleted directly via the AWS Console or CLI without requiring configuration modifications.
- Operational Status: Adequate for development environments; however, it exposes data to accidental loss or corruption risks if production-level mutations are introduced.

4 Security Assessment

4.1 IAM Least Privilege

AWS Managed Policy (CloudWatch Logging)

```
$ aws iam list-attached-role-policies --role-name dnt-nextjs-task-manager-crud-lambda-role-u6qddxbh
{
  "AttachedPolicies": [
    {
      "PolicyName": "AWSLambdaBasicExecutionRole-025017db-7b88-4329-b6f8-1e8ba54cf6d5",
      "PolicyArn": "arn:aws:iam::978872789817:policy/service-role/AWSLambdaBasicExecutionRole-025017db-7b88-4329-b6f8-1e8ba54cf6d5"
    }
  ]
}
```

Inline Policy (Database Access)

```
$ aws iam list-role-policies --role-name dnt-nextjs-task-manager-crud-lambda-role-u6qddxbh
{
  "PolicyNames": [
    "dnt-dynamodb-crud-policy"
  ]
}

$ aws iam get-role-policy --role-name dnt-nextjs-task-manager-crud-lambda-role-u6qddxbh --policy-name dnt-dynamodb-crud-policy
{
  "RoleName": "dnt-nextjs-task-manager-crud-lambda-role-u6qddxbh",
  "PolicyName": "dnt-dynamodb-crud-policy",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "dynamodb:Query",
          "dynamodb:PutItem",
          "dynamodb:UpdateItem",
          "dynamodb:DeleteItem"
        ],
        "Resource": "arn:aws:dynamodb:us-east-1:978872789817:table/dnt-nextjs-task-manager-tasks-table"
      }
    ]
  }
}
```

Conclusion

- Action Level: Restricted to the four required permissions (Query, PutItem, UpdateItem, DeleteItem). Broad permissions such as dynamodb:* and dynamodb:Scan were excluded. This prevents full-table scan operations, mitigating risks related to cost and performance.
- Resource Level: Avoided using Resource: "*". Access is explicitly scoped to the ARN of the dnt-nextjs-task-manager-tasks-table. This ensures that even if the Lambda function is compromised, it cannot access other DynamoDB tables within the AWS account.

4.2 Threat Modeling & STRIDE Assessment

A formal threat modeling assessment based on the STRIDE methodology was conducted focused on the Frontend deployment and Static Storage layers.

- Spoofing Identity
 - Threat Context: Malicious actors attempting to forge user identities or replay intercepted sessions to gain unauthorized access to the task management application.
 - Mitigation Implementation: The frontend layer strictly enforces user authentication via Amazon Cognito User Pool. Authenticated sessions are securely issued short-lived cryptographic JSON Web Tokens (JWT) directly to the client browser.
- Tampering
 - Threat Context: Malicious actors bypassing the Content Delivery Network (CDN) to directly inject scripts or modify static files inside the S3 bucket.
 - Mitigation Implementation: Origin Access Control (OAC) is enforced on the S3 bucket. The S3 bucket policy is configured to exclusively permit read operations originating from the designated Amazon CloudFront distribution ARN, rendering direct external manipulation impossible.
- Information Disclosure
 - Threat Context: Exposure of sensitive backend endpoint configurations, third-party administrative credentials, or direct raw access to application assets stored in S3.
 - Mitigation Implementation:
 - Client-Side Build Hygiene: The Next.js client-side source code is strictly audited to ensure zero administrative API keys, internal secrets, or database credentials are hardcoded within the deployed static build files.
 - Public Access Block Configuration: The storage layer enforces an absolute restriction on public access permissions.
 - Verifiable Evidence (S3 Public Access Block):

```
$ aws s3api get-public-access-block --bucket dnt-nextjs-task-manager-frontend
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}
```

- Elevation of Privilege
 - Threat Context: Authenticated low-privileged users attempting to invoke administrative backend endpoints directly, or unauthenticated clients executing remote backend tasks by bypassing frontend routers.
 - Mitigation Implementation: The Amazon API Gateway acts as an isolated security perimeter. Every incoming client request must attach a valid JWT within the authorization headers. The API Gateway automatically communicates with Amazon Cognito to validate the token's cryptographic signature, expiration status, and claims before forwarding the authorized payload to the execution Lambda layer.

4.3 API Protection & Rate Limiting

To safeguard the backend execution layer from application-layer application flooding (L7 DoS) and mitigate cloud resource over-consumption (Billing Attacks), strict rate limiting boundaries are enforced at the Amazon API Gateway tier using an API Usage Plan.

4.3.1 Current Security Configuration Status

The screenshot displays the AWS API Gateway console for the usage plan named "dnt-nextjs-task-manager-usage-plan". The console shows the following details:

- Usage plan details:**
 - Usage plan ID:** etxozp
 - Description:** Usage plan for Task Manager App to prevent Billing DDoS
 - AWS Marketplace product code:** -
 - Rate:** 10 requests per second
 - Burst:** 20 requests
 - Quota:** 1000 requests per day
- Associated stages:**
 - API: dnt-nextjs-task-manager-api
 - Stage: dev
 - Method throttling: -

At the top right, it indicates "Last updated: July 13, 2026, 10:33 (UTC+07:00)". There are buttons for "Actions", "Export usage data", "Edit", "Remove", and "Add stage".

4.3.2 Technical Security Analysis

Infrastructure-Level DDoS Protection: Large-scale network and transport layer attacks (L3/L4 DDoS) are automatically absorbed and mitigated by the native integration of AWS Shield Standard at the API Gateway edge.

Application-Level Rate Limiting (Throttling): The API Gateway layer enforces a steady-state rateLimit of 10.0 requests per second (rps) with a transient burstLimit of 20 requests. Any incoming burst traffic exceeding these limits is immediately rejected with a 429 Too Many Requests HTTP status code, shielding the downstream AWS Lambda function from execution spikes.

Resource & Cost Protection (Quota): An absolute volume perimeter is configured at 1,000 requests per day. This serves as a critical defense line against runaway client-side loops or malicious billing exploitation, ensuring predictable operational costs.

4.4 Static Application Security Testing (SAST) & Secure Code Review

To ensure both backend handler functions and frontend components comply with secure coding principles prior to deployment, a comprehensive static security review framework is established based on the STRIDE model.

4.4.1 Security Review Checklist & Vulnerability Mapping

STRIDE Category	Code Review Security Control (Self-Check)	Risk Impact if Exploited / Vulnerability Class
Tampering	Is proper input validation and contextual sanitization enforced on client-supplied datafields (e.g., Task Title, Content)?	Malicious payload injection leading to Stored/Reflected Cross-Site Scripting (XSS) within user sessions.
Information Disclosure	Are internal execution errors caught gracefully, ensuring raw system details are not returned back to the client?	Information Leakage / Stack Trace Exposure, allowing threat actors to map internal

		database schemas or code structures.
Denial of Service	Are system resource consumption and API call rates bounded?	Downstream resource exhaustion (Mitigated globally at the Amazon API Gateway layer as detailed in Section 3).
Elevation of Privilege	Is multi-tenant isolation rigorously verified at the database query level to prevent a user from mutating resources owned by another?	Broken Object Level Authorization (BOLA) / Insecure Direct Object References (IDOR), allowing unauthorized manipulation of tenant data.

4.4.2 Execution & Results

Status: Pending / In Progress

Next Steps: Integration of lightweight static analysis tooling (e.g., Semgrep or GitHub Actions security scanning) into the source code repository is planned for the next development sprint to automate the validation of non-logic code flaws.

5 Performance Capacity & Operational Costs

5.1 Performance Volumetric Assumptions & Metrics

5.1.1 User Behavior Profiles

To establish a baseline for capacity planning and Request Per Second (RPS) metrics, the daily interaction pattern for a single active user is assumed as follows:

- Each user executes an average of 10 transactional actions per day (consisting of 1x Login, 6x View Tasks, 1x Create Task, 1x Update Task, and 1x Delete Task).
- Given a baseline target of 100 active users, the system volume is calculated at: 100 users * 10 actions = 1,000 total requests/day.

5.1.2 Calculated Baseline Workload Metrics

Metric	Execution Logic / Calculation Formula	Estimated Operational Value
Total Daily Requests	100 users $\times$ 10 requests/user	1,000 requests/day
Peak Hour Traffic Volume	Assuming a 20% traffic concentration within 1 single peak hour	200 requests/hour
Throughput (Requests Per Second)	200 requests / 3,600 seconds	~0.06 RPS
Concurrent Users	Calculated via Little's Law ($RPS \times \text{Latency}$); given extremely low steady-state RPS	< 1 concurrent user
Data Storage Growth	100 users $\times$ 1 new task/day $\times$ 1 KB average metadata per item payload	~100 KB/day (~3 MB/month)

5.2 Empirical Load Test Results

Status: Pending Execution.

Note: Synthetic traffic generation scripts will be scheduled in subsequent operational phases to confirm theoretical limit thresholds.

5.3 Architectural Component Performance Analysis

Under a steady-state workload of ~0.06 RPS, the serverless architecture operates well within its comfort zone:

- AWS Lambda Layer: The compute workload is minimal, leaving the execution environment idle 99% of the time. While low transaction density means backend containers will be de-provisioned frequently (leading to cold starts), the native optimization of the Node.js 22.x runtime combined with a lightweight 128 MB RAM footprint ensures cold start overhead remains constrained within a negligible 100–300ms window.
- Amazon DynamoDB Layer: Operating under an On-Demand billing mode, the database seamlessly provisions throughput without manual management. Processing 1,000 requests per day generates nominal write/read units, resulting in an operational cost of only a few cents per month.
- Amazon API Gateway Tier: The baseline volume of 30,000 requests per month sits substantially below the free allocation tiers, delivering optimal response times with zero maintenance requirements.
- Potential Latency Bottlenecks: At this workload level, any end-user perceived latency will not stem from backend performance choke points. Potential causes are restricted to:
 - o Bloated frontend client-side rendering operations within the Next.js framework.
 - o Local transit network degradation at the end-user side.
 - o Sub-optimal CloudFront distribution caching behavior for static files.

5.4 Performance Review Assessment

Architecture Fit: The serverless tech stack is highly scalable for current needs. It provides a reliable foundation capable of sustaining a 10x to 100x increase in user traffic seamlessly without requiring infrastructure modifications or source code rewrites.

Cost Efficiency Summary: Base infrastructure spending is extremely optimized, projected at less than \$1.00 USD/month under normal operational profiles, assuming no anomalous logic iteration loops or external distributed service disruptions.

5.5 Cost Estimation & Financial Financial Risk Vectors

5.5.1 Monthly Cost Breakdown (Based on Baseline Workload)

Traffic Volume Assumption: 100 active users/day -> ~30,000 requests/month.

Pricing Model: Calculated based on the AWS Free Tier framework (applicable within the first 12 months of account activation, with several services covered under the Always-Free tier).

AWS Infrastructure Service	Projected Monthly Volume	Cost Calculation Basis / Free Tier Threshold	Estimated Monthly Cost (USD)
Amazon Route 53	1 Hosted Zone	Flat rate of \$0.50 per month per hosted zone (No Free Tier tier applicable).	\$0.50
Amazon API Gateway	30,000 requests	Well within the 1 Million requests/month allocation (Free Tier).	\$0.00
AWS Lambda	30,000 invocations	Covered by the 1 Million free requests & 400,000 GB-seconds per month (Always Free).	\$0.00
Amazon DynamoDB	30,000 database operations	Sits below the 25 GB storage and 25 WCU/RCU capacity allocation (Always Free).	\$0.00

Amazon S3	~50 MB static assets	Well within the 5 GB standard storage tier allowance.	\$0.00
TOTAL ESTIMATED COST: ~\$0.50 USD / Month			

5.5.2 Cost Anomalies & Financial Risk Mitigation

While the normal operational baseline is highly optimized at \$0.50/month, the following edge cases represent architectural risks that could trigger unexpected cost slippage:

- 12-Month Free Tier Expiration: Certain non-always-free services (such as standard Amazon S3 storage allocations) will transition to standard pay-as-you-go metrics after the initial 12-month account lifespan. Resource boundaries must be periodically reviewed to maintain low expenditure.
- Data Transfer Out Spikes (CloudFront): If an administrative user accidentally uploads oversized assets, or if the distribution endpoint faces an unmitigated application-layer asset scraping attack that pushes bandwidth consumption past the 1 TB monthly ceiling, standard network egress fees will be incurred.
- Unmonitored Auxiliary Cloud Services: Introducing observability features such as AWS X-Ray tracing, extensive Amazon CloudWatch Custom Metrics, or active CloudWatch Alarms without strict retention and filtering controls can easily generate unexpected operational line items.

6 Conclusion & Operational Action Plan

6.1 Comprehensive Architectural Pros & Cons Matrix

Engineering Vector	Architectural Advantages (Pros)	Architectural Limitations (Cons)
Operations (Ops)	Completely eliminates OS patching, compute server provisioning, and runtime capacity management. Entirely fully-managed infrastructure.	Higher debugging complexity. Lacks direct server boundary shell access (SSH); completely dependent on distributed Amazon CloudWatch execution logs.
Financial Cost	Pure "Pay-as-you-go" elasticity. Zero computational footprint generates zero operational cost, highly cost-effective for micro-to-medium systems.	Vulnerable to <i>Denial of Wallet (DoW)</i> or Layer 7 DDoS vectors. Anomalous runtime software implementation (unbounded iteration loops) can result in sudden pricing spikes.
Performance Scaling	Instantaneous horizontal auto-scaling triggered dynamically by incoming request spikes.	Intermittent Cold Starts: Infrequently invoked Lambda environments face up to a few hundred milliseconds latency overhead during initial micro-container provisioning.
Security Posture	Fine-grained native AWS IAM integration enforces strict compliance with the Principle of Least Privilege.	Overhead in configuration surface management. Single sub-optimal IAM or resource policy can lead to severe operational failure or security exposures.
Architectural Agility	Loosely coupled architecture simplifies modular enhancement or targeted upgrades of individual microservices.	Vendor Lock-in: Decoupling from native services (Lambda/DynamoDB) to alternative hyperscalers (Azure/GCP) necessitates a refactoring of persistence and compute interface layers.

6.2 Component-Level Evaluation

6.2.1 Amazon CloudFront + S3 Edge Layer

Strengths: Excellent response velocity and enhanced security provided by Origin Access Control (OAC) enforcement.

Identified Risk: Static file multi-tenant hosting vulnerability; accidental deployment of environmental parameter secrets or core backend codebase source repositories to the public S3 origin boundary without strict code review will result in permanent public information exposure.

6.2.2 Amazon API Gateway + AWS Lambda Layer

Strengths: Robust mitigation against application-layer DDoS traffic and financial cost overruns through strict execution throttling and Usage Plan quota constraints.

Identified Risk: Highly decoupled observability pattern. If native structured logging (CloudWatch) and distributed tracing (AWS X-Ray) features are not thoroughly instrumented, tracing multi-service transaction failures across runtime boundaries becomes intensely complex.

6.2.3 Amazon Cognito User Pool

Strengths: Enterprise-grade security handling identity management, user enrollment, and session authorization tokens.

Identified Risk: Rigidity in identity routing rules. Implementing custom authentication patterns requires adding Cognito Lambda Triggers, which significantly increases code maintenance overhead.

6.3 Targeted Improvement Action Items

To transition the architecture to production-grade status, the following technical actions are prioritized for execution:

- Cold Start Optimization: Minimize AWS Lambda dependency footprints by trimming Node.js production bundle outputs, applying tree-shaking patterns, and utilizing highly optimized, lightweight third-party utility modules.
- Observability Enrichment: Enable and configure AWS X-Ray agents alongside structured logger utilities to trace end-to-end request transaction flows across the API Gateway and serverless execution spaces.
- Mitigating Vendor Lock-in: Enforce a strict decoupling of business domain logic from core cloud providers. Isolate the operational business logic from the native AWS Lambda handler event payload dependencies using clean architectural separation.